LMA Individual Project — Phase 1 Report

Data Collection and Tokenizer Construction
Hindi (Model H) and Nepali (Model L)

Gopal
2023112006

August 19, 2026

1 Overview

This report documents the Phase 1 data collection, preprocessing, splitting, and tokenizer construction pipeline for two independent monolingual language models: a higher-resource Hindi model (**Model H**) and a lower-resource Nepali model (**Model L**). The two corpora, tokenizer training runs, vocabularies, and evaluation artifacts are kept separate by language. No pretrained tokenizer or shared vocabulary is used.

2 Language Selection

2.1 Model H: Hindi

Hindi was selected as the higher-resource language because it has a large public digital footprint across news, literature, public-domain books, magazines, and educational material. This made it a suitable anchor language for Phase 1: enough data could be collected from many independent sources, and the author's familiarity with Hindi will also support qualitative generation checks in later phases.

2.2 Model L: Nepali

Nepali was selected from the permitted low-resource language list. It is written primarily in Devanagari, which allowed the same script-aware crawler, cleaner, and tokenizer infrastructure to be reused while still testing the pipeline on a language that is lower-resource than Hindi. Because Hindi and Nepali share a script, the corpus separation is enforced at the source level: Hindi data is drawn from Hindi-specific domains and collections, while Nepali data is drawn from Nepali domains and the Nepali portion of IndicCorp v2.

3 Data Collection

3.1 Custom Crawler

Manual/self-collected data was gathered with the custom crawler in `crawler/`. The crawler performs recursive in-domain link traversal from seed URLs, keeps a persistent SQLite frontier and visited-state database, stores extracted document hashes, and supports resumable crawling across

sessions. The crawl policy used conservative throttling: up to 3 requests per second per source configuration, bounded concurrency, retry limits, and request timeouts. Sitemap and RSS fields are present in the source configuration, and the crawler also supports PDF-oriented collection through the surrounding extraction scripts.

HTML extraction uses `trafilatura` where available, with BeautifulSoup/lxml-based parsing and cleanup as fallbacks. The extractor removes scripts, styles, navigation residue, and repeated boilerplate before the text is sent to the cleaning and filtering pipeline.

3.2 Sources

The Hindi corpus combines self-collected web and literary data with high-quality public domain collections. The largest Hindi self-crawled and literary sources include `hindwi`, `matrubharti_hindi`, `storymirror_hindi`, `femina_hindi`, `chakmak`, `meri_saheli`, `bbc_hindi`, `abplive_hindi`, `punjab_kesari`, and `abhivyakti_hindi`.

The Nepali corpus combines a large downloaded public corpus component with self-crawled Nepali web data. The downloaded component is the Nepali portion of IndicCorp v2, recorded in the dataset metadata as `indiccorp_v2`. The largest Nepali self-crawled sources include `onlinekhabar`, `hamrokhelkud`, `shirishnews`, `nepallive`, `ekantipur`, `swasthyakhabar`, `setopati`, `gorkhapatra`, `arthakendra`, and `deshsanchar`.

Table 1: Manual and downloaded token split

Language	Total tokens	Manual tokens	Downloaded tokens	Manual share
Hindi	500,000,757	100,000,757	400,000,000	20.00%
Nepali	703,417,302	110,106,774	593,310,528	15.65%

The Hindi corpus satisfies the assignment's 20% manual collection threshold (100.0M tokens self-collected out of 500.0M total). The Nepali corpus exceeds the 500M-token target after adding IndicCorp v2, with a token-weighted manual share of 15.65% (110.1M self-collected tokens).

4 Cleaning Pipeline

The same cleaning design was used for both languages:

1. Unicode text is normalized to NFC before filtering and hashing.
2. HTML boilerplate is removed with `trafilatura` and fallback BeautifulSoup/lxml parsing.
3. Documents with insufficient Devanagari character coverage are rejected. The crawler configs use source-specific thresholds, and the split/statistics scripts retain only Devanagari-heavy documents.
4. Language separation is enforced by source selection, because a Devanagari ratio alone cannot distinguish Hindi from Nepali.
5. Exact duplicates are removed with SHA-256 hashes of normalized text.
6. Near duplicates are filtered with MinHash/LSH over word shingles in the cleaning pipeline.

7. Very short fragments are removed to suppress navigation snippets, empty pages, captions, and extraction failures.

The current repository preserves final dataset statistics, source distributions, and train/validation/test splits.

5 Corpus Statistics

Table 2: Final corpus statistics

Language	Docs	Words	Est. BPE tokens	UTF-8 MB	Avg. words/doc	Avg. Devanagari
Hindi	5,390,540	404,204,331	500,000,757	5014.94	74.98	77.51%
Nepali	13,213,090	530,080,861	703,417,302	8735.29	40.12	83.54%

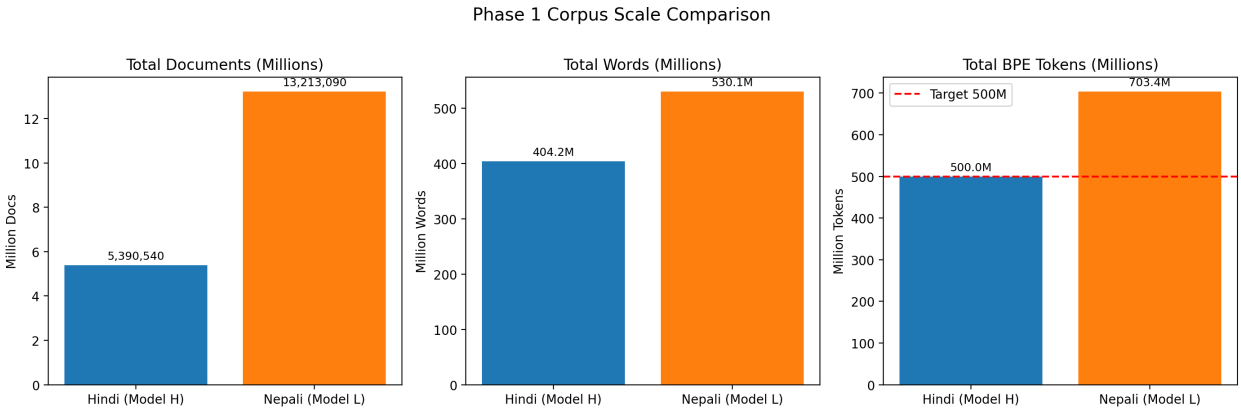


Figure 1: Corpus size comparison by documents, words, and estimated BPE tokens.

Table 3: Document length statistics in words

Language	Min	P50	Mean	P90	P95	P99	Max
Hindi	1	38	74.98	142	231	672	245087
Nepali	1	29	40.12	67	90	234	15776

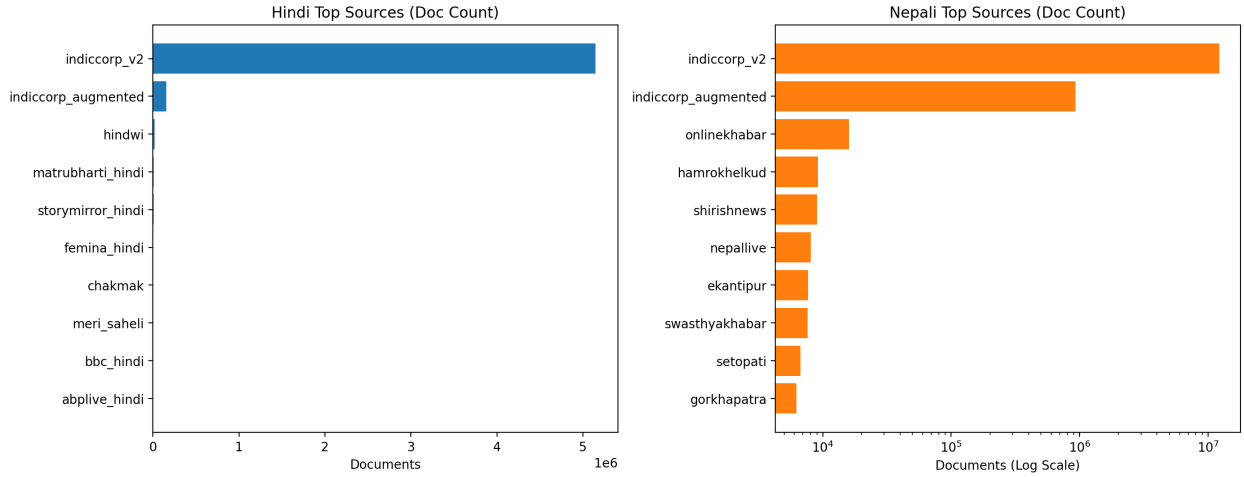


Figure 2: Top source distribution by document count.

6 Train, Validation, and Test Splits

Splits are document-level, not sentence-level, to avoid leakage from the same article or book segment appearing in multiple splits. The split ratio is 90% train, 5% validation, and 5% test with a fixed random seed in the preprocessing scripts.

Table 4: Split sizes by language

Language	Split	Docs	Words	Est. BPE tokens
Hindi	Train	4,851,486	363,528,216	449,684,403
Hindi	Validation	269,527	20,495,877	25,353,399
Hindi	Test	269,527	20,180,238	24,962,954
Nepali	Train	11,891,781	476,919,617	632,872,331
Nepali	Validation	660,654	26,564,857	35,251,565
Nepali	Test	660,655	26,596,387	35,293,405

7 Tokenizer Training

7.1 Method

A separate custom BPE tokenizer was trained from scratch for each language using `tokenizer/tokenizers.py`. The implementation uses a Devanagari-aware regex pre-tokenizer, initializes words as character sequences with an end-of-word marker, counts adjacent token pairs weighted by word frequency, and repeatedly merges the highest-frequency pair using a heap-backed pair index. Training stops when the requested merge budget is reached or when no pair remains above the minimum frequency threshold. The saved artifacts include a pickle model, vocabulary JSON, merges text file, and config JSON for each language and vocabulary budget.

7.2 Vocabulary Sweep

Candidate BPE budgets of 5K, 10K, and 20K merges were evaluated on held-out test splits. The realized vocabulary size can differ from the nominal merge budget because the implementation begins with a character-level base vocabulary and stops according to pair-frequency availability.

Table 5: Tokenizer sweep

Language	Tokenizer	Vocab	Fertility	Chars/token	UNK %	Tok/s
Hindi	BPE-5K	6832	1.55	3.27	0.0	1,193,593
Hindi	BPE-10K	11263	1.397	3.63	0.0	1,060,313
Hindi	BPE-20K	20195	1.29	3.93	0.0	979,965
Nepali	BPE-5K	4777	1.731	3.73	0.0	1,019,097
Nepali	BPE-10K	9137	1.507	4.28	0.0	892,769
Nepali	BPE-20K	17973	1.353	4.77	0.0	791,549

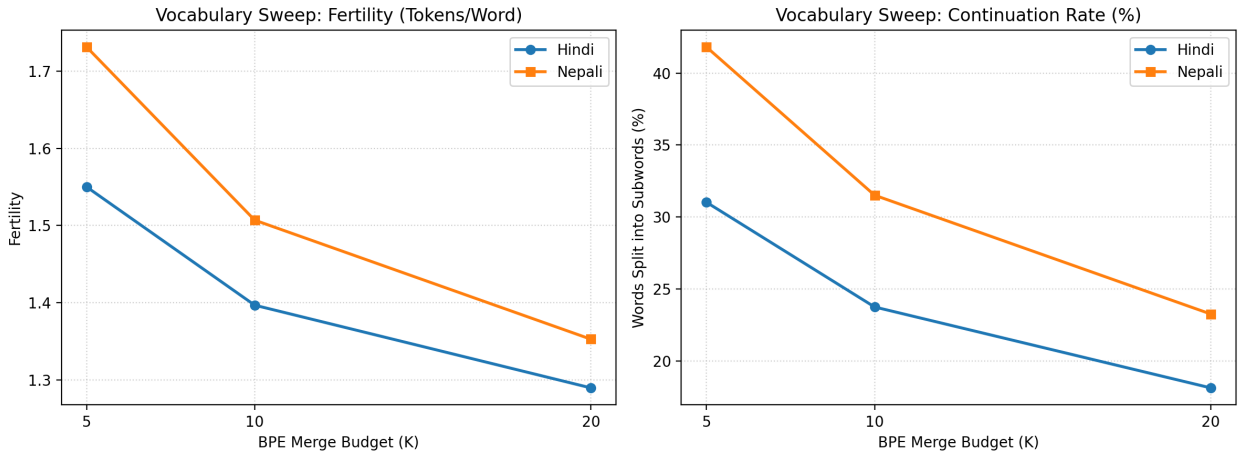


Figure 3: Effect of BPE merge budget on fertility and unknown-token rate.

The 20K-budget tokenizers are selected for the next phase: `bpe_hi_20000` for Hindi and `bpe_ne_20000` for Nepali. They have the lowest fertility among the evaluated settings while preserving 0.00% unknown-token rate. Hindi reaches 1.290 tokens/word with a 20,195-token vocabulary; Nepali reaches 1.353 tokens/word with a 17,973-token vocabulary.

7.3 Tokenization Examples

The examples below use the selected 20K-budget tokenizers.

Table 6: Hindi tokenization examples

Input	BPE pieces
भारतीय संविधान दुनिया का सबसे लंबा लिखित संविधान है।	भार | तीय | संविधान | दुनिया | का | सबसे | लंबा | लिखि | त | संविधान | है।
भाषा मॉडल प्राकृतिक भाषा प्रसंस्करण और कृत्रिम बुद्धिमत्ता में महत्वपूर्ण भूमिका निभाते हैं।	भाषा | मॉडल | प्राकृतिक | भाषा | प्र | संस् | करण | और | कृ | त्रिम | बुद्धिमत्ता | में | महत्वपूर्ण | भूमिका | निभाते | हैं।
प्रेमचंद की कहानियों में भारतीय ग्रामीण जीवन की वास्तविक तस्वीर देखने को मिलती है।	प्रेमचंद | की | कहानियों | में | भार | तीय | ग्राम | ईण | जीवन | की | वास् | तविक | तस्वीर | देखने | को | मिलती | है।

Table 7: Nepali tokenization examples

Input	BPE pieces
नेपाल एक बहुजातीय, बहुभाषिक, बहुधार्मिक र बहुसांस्कृतिक विशेषतायुक्त मुलुक हो।	नेपाल | एक | बहु | जातीय | , | बहु | भाषिक | , | बहु | धार्मिक | र | बहु | सांस्कृतिक | विशेष | ता | युक्त | मुलुक | हो।
सगरमाथा संसारको सबैभन्दा अग्लो हिमशिखर हो जुन नेपालको सोलुखुम्बु जिल्लामा पर्छ।	सगरमाथा | संसारको | सबैभन्दा | अग्लो | हिम | शिखर | हो | जुन | नेपालको | सोलुखु | म्बु | जिल्लामा | पर्छ।
कृत्रिम बौद्धिकता र भाषा प्रविधिले मानव जीवनलाई सहज र प्रभावकारी बनाएको छ।	कृत्रिम | बौद्धि | कता | र | भाषा | प्रविधिले | मानव | जीवनलाई | सहज | र | प्रभावकारी | बनाएको | छ।

7.4 Token Frequency

The 20K-budget tokenizers produce language-appropriate high-frequency tokens. Hindi's most frequent held-out tokens include punctuation and common postpositions such as के, में, की, से, and है. Nepali's most frequent tokens include ।, comma, र, छ, पनि, को, भएको, and मा. On the test splits, the top 100 token types cover 40.17% of Hindi tokens and 25.99% of Nepali tokens.

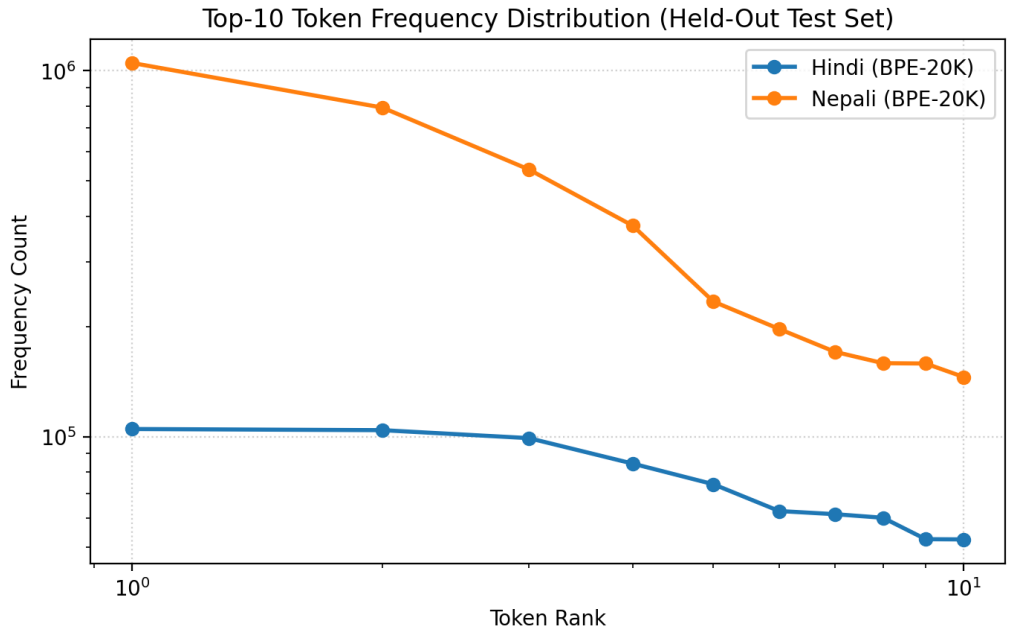


Figure 4: Token frequency distribution on held-out test splits.

8 Summary

Phase 1 produced independent Hindi and Nepali datasets, reproducible document-level splits, source/statistics metadata, trained BPE artifacts, and held-out tokenizer evaluations. The main technical risks for later phases are the Nepali manual-share shortfall and the shared-script language separation problem, both of which are visible in the report and should be addressed by additional Nepali crawling or stricter lexical language identification before final model training. The selected 20K-budget BPE tokenizers are ready for Phase 2 pretraining.